

Objetivo

Auditar los permisos efectivos de una ServiceAccount mediante `kubectl auth can-i`. Diagnosticar una autorización faltante sin ampliar innecesariamente el alcance del Role. Aplicar el principio de mínimo privilegio agregando únicamente lectura sobre Services. Validar desde impersonación y desde un Pod real que los permisos sensibles continúan denegados.

Requisitos previos

- Haber completado la Práctica 5 y conservar el namespace `lab5`.
- Conservar la ServiceAccount `app-reader` y el RoleBinding `app-reader-binding`.
- Tener Docker Desktop, Minikube y `kubectl` operativos.
- Utilizar Visual Studio Code con Git Bash como terminal integrada.
- Disponer del archivo `rbac-scenario5-1.yaml` proporcionado para este reto.

Duración estimada

20 minutos

Introducción

En este reto recibirás una identidad de aplicación que puede consultar Pods y ConfigMaps, pero una nueva función de descubrimiento de Services falla con autorización denegada. No conocerás inicialmente qué permiso falta. Deberás reproducir el problema, inspeccionar el modelo RBAC efectivo, aplicar la corrección mínima y demostrar que Secrets, escritura y otros namespaces continúan protegidos.

Instrucciones



Escenario del reto

La ServiceAccount `app-reader` fue creada en la Práctica 5 para permitir consultas de lectura sobre Pods y ConfigMaps dentro del namespace `lab5`.

El equipo de desarrollo agregó una función que necesita descubrir los Services disponibles en ese mismo namespace. La aplicación ahora recibe una respuesta de autorización denegada cuando intenta consultar esos recursos.

Tu objetivo es determinar:

1. ¿Qué permisos tiene actualmente app-reader?
2. ¿Qué operación relacionada con Services está siendo denegada?
3. ¿Qué regla falta en el Role?
4. ¿Cuál es la corrección mínima necesaria?
5. ¿Cómo demostrar que los permisos sensibles continúan bloqueados?

 Copiar

Distribución sugerida

Preparación	2 min
Reto 1	6 min
Reto 2	7 min
Reto 3	5 min
Total	20 min

 Copiar

Preparación del escenario

Confirmar los recursos reutilizados

- **Paso 1.** Verifica que el namespace `lab5` continúa activo, porque todos los objetos RBAC de la práctica principal fueron creados dentro de ese ámbito.

```
kubectl get namespace lab5
```

 Copiar

Salida esperada aproximada:

```
NAME    STATUS   AGE
lab5    Active   ...
```

 Copiar

- **Paso 2.** Comprueba que la ServiceAccount y el RoleBinding siguen disponibles antes de modificar cualquier permiso.

```
kubectl get serviceaccount app-reader \
-n lab5

kubectl get rolebinding app-reader-binding \
-n lab5
```

 Copiar

Salida esperada aproximada:

NAME	SECRETS	AGE
app-reader	0	...

NAME	ROLE	AGE
app-reader-binding	Role/pod-configmap-reader	...

 Copiar

NOTA: En versiones actuales de Kubernetes es normal que la columna **SECRETS** de una ServiceAccount muestre **0**.

Aplicar el estado inicial del reto

- **Paso 3.** Descarga `rbac-scenario5-1.yaml`; el archivo restablece únicamente el Role utilizado en el ejercicio para garantizar un punto de partida conocido.

```
curl -L \
-o rbac-scenario5-1.yaml \
https://raw.githubusercontent.com/Netec-
Mx/CKA/refs/heads/main/labs/lab9/rbac-scenario5-
1.yaml
```

 Copiar

IMPORTANTE: No abras ni modifiques el archivo antes de completar el primer reto. La causa debe identificarse mediante los permisos efectivos de Kubernetes.

- **Paso 4.** Aplica el escenario para dejar `pod-configmap-reader` en el estado base esperado antes de iniciar el diagnóstico.

```
kubectl apply -f rbac-scenario5-1.yaml
```

 Copiar**Salida esperada:**

```
service/discovery-api created  
role.rbac.authorization.k8s.io/pod-configmap-reader  
unchanged
```

 Copiar

La salida también puede mostrar `created` si el Role no existía.

Reto 1. Auditar los permisos efectivos de app-reader

Tiempo sugerido: 6 minutos

En este reto no debes modificar RBAC. Primero construye evidencia suficiente para demostrar qué operaciones están permitidas y cuáles están bloqueadas.

Reto 1.1. Comprobar permisos conocidos

- **Paso 5.** Verifica que `app-reader` conserva lectura de Pods, confirmando que la identidad y su RoleBinding funcionan antes de investigar el nuevo requerimiento.

```
kubectl auth can-i get pods \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=lab5
```

 Copiar**Salida esperada:**

```
yes
```

 Copiar

- **Paso 6.** Comprueba también lectura de ConfigMaps para validar otra autorización definida originalmente en `pod-configmap-reader`.

```
kubectl auth can-i get configmaps \
  --as=system:serviceaccount:lab5:app-reader \
  --namespace=lab5
```

 Copiar

Salida esperada:

```
yes
```

 Copiar

Reto 1.2. Reproducir el nuevo problema

- **Paso 7.** Consulta si la identidad puede obtener Services dentro de `lab5`; esta prueba reproduce directamente la operación requerida por la nueva funcionalidad.

```
kubectl auth can-i get services \
  --as=system:serviceaccount:lab5:app-reader \
  --namespace=lab5
```

 Copiar

- **Paso 8.** Comprueba también `list services`, porque una función de descubrimiento normalmente necesita enumerar los Services disponibles y no solo consultar uno conocido.

```
kubectl auth can-i list services \
  --as=system:serviceaccount:lab5:app-reader \
  --namespace=lab5
```

 Copiar

Salida esperada inicial:

```
no
no
```

 Copiar

Reto 1.3. Verificar límites sensibles

- **Paso 9.** Confirma que la ServiceAccount no puede leer Secrets, estableciendo una línea base de seguridad que deberá conservarse después de la corrección.

```
kubectl auth can-i get secrets \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=lab5
```

 Copiar

- **Paso 10.** Comprueba que tampoco puede crear Pods, demostrando que el Role actual mantiene separados los permisos de lectura y escritura.

```
kubectl auth can-i create pods \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=lab5
```

 Copiar

- **Paso 11.** Verifica que un Role de `lab5` no otorga acceso equivalente sobre Pods de `kube-system`.

```
kubectl auth can-i get pods \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=kube-system
```

 Copiar

Salida esperada:

```
no  
no  
no
```

 Copiar

Evidencia requerida

Debes poder construir esta matriz:

OPERACIÓN	RESULTADO
get pods en lab5	yes
get configmaps en lab5	yes
get services en lab5	no
list services en lab5	no
get secrets en lab5	no
create pods en lab5	no
get pods en kube-system	no

 Copiar

Reto finalizado

Resultado esperado: Nuevo Service del escenario identificado y permisos actuales de `app-reader` auditados correctamente, comprobando que el acceso a Services está inicialmente denegado.



Reto 2. Identificar y corregir la autorización faltante

Tiempo sugerido: 7 minutos

Ahora debes localizar la regla responsable y aplicar solamente los permisos requeridos por la nueva función.

Reto 2.1. Inspeccionar el Role efectivo

- **Paso 12.** Describe `pod-configmap-reader` para identificar los recursos y verbos actualmente autorizados sin depender de una copia local del manifiesto.

```
kubectl describe role pod-configmap-reader \
-n lab5
```

[Copiar](#)

Salida esperada aproximada antes de corregir:

Resources	Verbs
-----	-----
Pods	[get list watch]
Pods/log	[get]
ConfigMaps	[get list watch]

[Copiar](#)

- **Paso 13.** Consulta el YAML efectivo para relacionar cada regla con `apiGroups`, `resources` y `verbs`.

```
kubectl get role pod-configmap-reader \  
-n lab5 \  
-o yaml
```

 Copiar

Reto 2.2. Formular la corrección

Antes de editar, determina qué nueva regla cumple simultáneamente:

```
[ ] Permite get services.  
[ ] Permite list services.  
[ ] Permite watch services.  
[ ] No permite create services.  
[ ] No permite update services.  
[ ] No permite delete services.  
[ ] No concede acceso a Secrets.  
[ ] Solo aplica dentro de lab5.
```

 Copiar

La corrección debe utilizar:

```
apiGroup principal de Kubernetes  
resource específico requerido  
verbos exclusivamente de lectura
```

 Copiar

Reto 2.3. Corregir el manifiesto del escenario

- **Paso 14.** Abre `rbac-scenario5-1.yaml` únicamente después de haber formulado el diagnóstico y agrega la regla mínima necesaria para Services.

```
code rbac-scenario5-1.yaml
```

 Copiar

IMPORTANTE: No reemplaces los recursos o verbos existentes por `*`. Tampoco cambies el Role por ClusterRole.

- **Paso 15.** Valida la sintaxis del archivo corregido antes de modificar el Role almacenado en Kubernetes.

```
kubectl apply \  
  --dry-run=client \  
  -f rbac-scenario5-1.yaml
```

 Copiar

Salida esperada:

```
role.rbac.authorization.k8s.io/pod-configmap-reader  
configured (dry run)
```

 Copiar

- **Paso 16.** Aplica el manifiesto corregido para actualizar exclusivamente las reglas del Role dentro de `lab5`.

```
kubectl apply -f rbac-scenario5-1.yaml
```

 Copiar

Salida esperada:

```
role.rbac.authorization.k8s.io/pod-configmap-reader  
configured
```

 Copiar

Reto 2.4. Confirmar la configuración efectiva

- **Paso 17.** Describe nuevamente el Role y confirma que ahora aparece una regla de lectura para `services` sin alterar las reglas existentes.

```
kubectl describe role pod-configmap-reader \  
  -n lab5
```

 Copiar

Salida esperada aproximada después de corregir:

Resources	Verbs
-----	-----
pods	[get list watch]
pods/log	[get]
configmaps	[get list watch]
services	[get list watch]

 Copiar

Reto finalizado

Resultado esperado: Role `pod-configmap-reader` corregido con `get`, `list` y `watch` sobre Services, aplicando mínimo privilegio sin modificar los permisos existentes.

✓ Reto 3. Validar mínimo privilegio desde Kubernetes y un Pod real

Tiempo sugerido: 5 minutos

La corrección no queda terminada hasta comprobar que resuelve el requerimiento sin introducir permisos adicionales.

Reto 3.1. Validar la nueva autorización

- **Paso 18.** Comprueba `get`, `list` y `watch` sobre Services utilizando impersonación de la ServiceAccount.

```
for verb in get list watch; do
  kubectl auth can-i "$verb" services \
    --as=system:serviceaccount:lab5:app-reader \
    --namespace=lab5
done
```

[Copiar](#)

Salida esperada:

```
yes
yes
yes
```

[Copiar](#)

- **Paso 19.** Verifica inmediatamente que las operaciones de escritura sobre Services continúan bloqueadas.

```
for verb in create update delete; do
  kubectl auth can-i "$verb" services \
    --as=system:serviceaccount:lab5:app-reader \
    --namespace=lab5
done
```

 Copiar

Salida esperada:

```
no
no
no
```

 Copiar

Reto 3.2. Volver a comprobar permisos sensibles

- **Paso 20.** Confirma que la ampliación del Role no concedió accidentalmente lectura de Secrets ni creación de Pods.

```
kubectl auth can-i get secrets \
  --as=system:serviceaccount:lab5:app-reader \
  --namespace=lab5

kubectl auth can-i create pods \
  --as=system:serviceaccount:lab5:app-reader \
  --namespace=lab5
```

 Copiar

Salida esperada:

```
no
no
```

 Copiar

- **Paso 21.** Confirma nuevamente que el Role sigue limitado al namespace lab5.

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:lab5:app-reader \
  --namespace=kube-system
```

 Copiar

Salida esperada:

no

 Copiar**Reto 3.3. Validar desde rbac-demo-pod**

- **Paso 22.** Comprueba si el Pod de validación de la práctica principal continúa disponible y confirma que utiliza la ServiceAccount correcta.

```
kubectl get pod rbac-demo-pod \
-n lab5

kubectl get pod rbac-demo-pod \
-n lab5 \
-o jsonpath='{.spec.serviceAccountName}'
```

 Copiar**Salida esperada:**

app-reader

 Copiar

NOTA: Si `rbac-demo-pod` no existe, vuelve a aplicar `pod-rbac.yaml` del Lab 8 antes de continuar con esta comprobación.

- **Paso 23.** Ejecuta `kubectl get services` desde el Pod para demostrar que el token real de `app-reader` recibe ahora la nueva autorización.

```
kubectl exec rbac-demo-pod \
-n lab5 -- \
kubectl get services -n lab5
```

 Copiar**Salida esperada aproximada:**

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
PORT(S)	AGE		
...			

 Copiar

La lista puede variar según los Services existentes en `lab5`; lo importante es que la operación ya no responda `Forbidden`.

- **Paso 24.** Intenta listar Secrets desde el mismo Pod para confirmar que la API continúa bloqueando el recurso sensible.

```
kubectl exec rbac-demo-pod \  
-n lab5 -- \  
sh -c 'kubectl get secrets -n lab5 || true'
```

[Copiar](#)

Salida esperada aproximada:

```
Error from server (Forbidden): secrets is forbidden:  
User "system:serviceaccount:lab5:app-reader" cannot  
list resource "secrets" ...
```

[Copiar](#)

Reto finalizado

Resultado esperado: Acceso a Services validado correctamente después del ajuste, manteniendo denegados Secrets, escritura sobre Services y acceso fuera del namespace `lab5`.

✓ Validación final del reto

- **Paso 25.** Ejecuta el bloque siguiente para obtener una comprobación resumida del estado final de autorización de `app-reader`.

```
echo "=== Validacion complementaria 5.1 ==="

echo ""
echo "--- Lectura de Services ---"
for verb in get list watch; do
    RESULT=$(kubectl auth can-i "$verb" services \
        --as=system:serviceaccount:lab5:app-reader \
        --namespace=lab5)

    echo "$verb services: $RESULT"
done

echo ""
echo "--- Escritura de Services ---"
for verb in create update delete; do
    RESULT=$(kubectl auth can-i "$verb" services \
        --as=system:serviceaccount:lab5:app-reader \
        --namespace=lab5)

    echo "$verb services: $RESULT"
done

echo ""
echo "--- Permisos sensibles ---"
echo "get secrets: $(kubectl auth can-i get
secrets \
    --as=system:serviceaccount:lab5:app-reader \
    --namespace=lab5)"

echo "create pods: $(kubectl auth can-i create
pods \
    --as=system:serviceaccount:lab5:app-reader \
    --namespace=lab5)"

echo "get pods kube-system: $(kubectl auth can-i
get pods \
    --as=system:serviceaccount:lab5:app-reader \
    --namespace=kube-system)"

echo ""
echo "=== Fin de validacion ==="
```

Salida esperada:

```
=== Validacion complementaria 5.1 ===
```

 Copiar

```
--- Lectura de Services ---
```

```
get services: yes  
list services: yes  
watch services: yes
```

```
--- Escritura de Services ---
```

```
create services: no  
update services: no  
delete services: no
```

```
--- Permisos sensibles ---
```

```
get secrets: no  
create pods: no  
get pods kube-system: no
```

```
=== Fin de validacion ===
```

Condiciones para superar el reto

```
[✓] app-reader conserva lectura de Pods y  
ConfigMaps.
```

 Copiar

```
[✓] app-reader puede get, list y watch Services en  
lab5.
```

```
[✓] app-reader no puede crear, actualizar ni eliminar  
Services.
```

```
[✓] app-reader no puede leer Secrets.
```

```
[✓] app-reader no puede crear Pods.
```

```
[✓] app-reader no obtiene permisos equivalentes en  
kube-system.
```

```
[✓] rbac-demo-pod utiliza app-reader.
```

```
[✓] La consulta real de Services desde el Pod  
funciona.
```

IMPORTANTE: Conserva el Role actualizado, el RoleBinding, la ServiceAccount y el namespace `lab5` para las siguientes prácticas.

Resolución de problemas

Problema 1. get services continúa devolviendo no

Síntoma: Después de actualizar el Role, la autorización sobre Services continúa denegada.

Análisis: Comprueba que la regla fue realmente almacenada en `pod-configmap-reader` y no únicamente editada en el archivo local.

```
kubectl get role pod-configmap-reader \
  -n lab5 \
  -o yaml

kubectl describe role pod-configmap-reader \
  -n lab5
```

[Copiar](#)

La configuración efectiva debe contener `services` con `get`, `list` y `watch`.

Problema 2. app-reader puede crear o eliminar Services

Síntoma: Alguna operación de escritura devuelve `yes`.

Análisis: El Role posee verbos adicionales o existe otra vinculación que amplía los permisos de la ServiceAccount.

```
kubectl auth can-i --list \
  --as=system:serviceaccount:lab5:app-reader \
  --namespace=lab5

kubectl get rolebindings \
  -n lab5
```

[Copiar](#)

No elimines bindings hasta identificar cuál concede el permiso.

Problema 3. app-reader puede leer Secrets

Síntoma: `kubectl auth can-i get secrets` devuelve `yes`.

Análisis: La regla creada en esta práctica no debe contener Secrets. Revisa permisos acumulados y posibles bindings adicionales.

```
kubectl auth can-i --list \  
  --as=system:serviceaccount:lab5:app-reader \  
  --namespace=lab5  
  
kubectl get rolebindings \  
  -n lab5 \  
  -o wide
```

 Copiar

Problema 4. rbac-demo-pod no existe

Síntoma: La validación desde un Pod real devuelve `NotFound`.

Solución: Regresa al directorio del Lab 8 y aplica nuevamente el manifiesto utilizado en la práctica principal.

```
cd /c/LABS/kubernetes/lab5  
  
kubectl apply -f pod-rbac.yaml  
  
kubectl wait \  
  --for=condition=Ready \  
  pod/rbac-demo-pod \  
  -n lab5 \  
  --timeout=90s
```

 Copiar

Después repite las comprobaciones del Reto 3.3.

Problema 5. La consulta desde rbac-demo-pod sigue mostrando Forbidden

Síntoma: La impersonación con `kubectl auth can-i` devuelve `yes`, pero el Pod no puede listar Services.

Análisis: Confirma que el Pod realmente utiliza `app-reader` y que ejecuta la consulta en `lab5`.

```
kubectl get pod rbac-demo-pod \
-n lab5 \
-o jsonpath='{.spec.serviceAccountName}'{"\n"}'

kubectl exec rbac-demo-pod \
-n lab5 -- \
kubectl auth can-i get services -n lab5
```

 Copiar

La identidad debe ser `app-reader` y la segunda consulta debe devolver `yes`.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar el reto habrás diagnosticado una autorización faltante en un Role, ampliado exclusivamente la lectura de Services dentro de lab5 y validado que la ServiceAccount app-reader conserva un modelo de mínimo privilegio.

Notas y consideraciones

- Esta práctica dura 20 minutos y reutiliza los recursos creados en el Lab 8.
- El namespace correcto de los recursos reutilizados es lab5.
- No utilices ClusterRole, ClusterRoleBinding, wildcards ni permisos de escritura para resolver el reto.
- No agregues acceso a Secrets.
- Esta práctica complementaria no incluye prompts de apoyo con IA.

URL de referencia

- [RBAC Authorization](#)
- [kubectl auth can-i](#)
- [Service Accounts](#)